

APUNTE ACTIVIDAD 4

Duck typing, Protocol, los regalos de Python y el cierre

Índice

6 Duck typing

Donde el diseño realmente cambia

- Pero esto NO invalida tu modelo UML
- Protocol: el contrato sin el acoplamiento

7 Regalos que Java no tiene

dataclass, herencia múltiple, operadores

- dataclass
- Herencia múltiple real
- Otros regalos

8 Checklist de desintoxicación

Los 7 java-ismos a buscar en tu código Python

Cierre · Qué es UML y qué es sintaxis

El experimento controlado de los dos manuales

6 Duck typing

Donde el diseño realmente cambia

Todo lo anterior es **traducción**. Esto es **rediseño**.

El nombre viene del dicho: si camina como un pato y hace cuac como un pato, es un pato. Aplicado al código: si el objeto tiene el método que necesito, me sirve, y no me importa de qué clase es ni de quién hereda.

El Ejercicio 7 de la serie usa lo que el manual llama «la superclase deducida»: el enunciado no menciona ninguna clase padre, pero al analizarlo se deduce que hay un concepto común y se lo modela. En Java, esa superclase es **obligatoria** - sin ella no hay tipo común, sin tipo común no hay lista que los contenga, y sin lista no hay polimorfismo. El compilador exige un ancestro compartido antes de dejarte tratar dos objetos igual.

En Python, el polimorfismo no necesita la herencia:

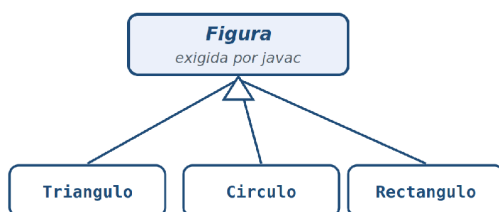
PYTHON

```
\# Sin superclase común: si tiene calcular_area(), sirve
for f in [Triangulo(), Circulo(), Rectangulo()]:
    print(f.calcular_area())
```

Esas tres clases pueden no tener absolutamente nada en común. Pueden venir de tres librerías distintas, escritas por gente que no se conoce. Mientras las tres respondan a `calcular_area()`, el bucle funciona. No hay declaración, no hay implements, no hay registro.

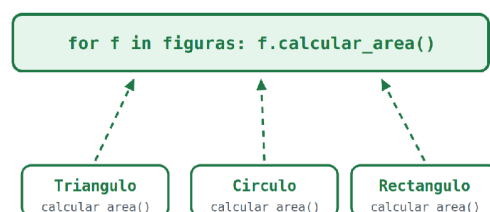
El polimorfismo, con y sin superclase

JAVA · el tipo común es obligatorio



sin ancestro común no hay `List<Figura>`,
y sin la lista no hay polimorfismo

PYTHON · si tiene el método, sirve



sin superclase, sin implements, sin registro:
el contrato es tener el método

Figura 8 · A la izquierda, la superclase que javac exige para el tipo común. A la derecha, el bucle que solo pide que el método exista.

Pero esto NO invalida tu modelo UML

Y acá está la lección central del capítulo, la que hay que resistirse a simplificar.

El reflejo, al descubrir el duck typing, es concluir que la herencia sobra y que las jerarquías del manual eran andamiaje del compilador. Es una conclusión apresurada. La superclase deducida del Ejercicio 7 no está ahí *solo* para que compile: está ahí porque **el dominio dice que esas cosas son un mismo concepto**. El UML modela el dominio, no las restricciones de javac. La superclase se justifica por semántica, no por sintaxis.

La diferencia es que en Java las dos razones -«el dominio lo dice» y «sin esto no compila»- producen exactamente el mismo código, y por lo tanto se confunden. Nunca tenés que elegir cuál era. En Python se separan: la razón sintáctica desaparece y solo queda la semántica. Y esa separación es un regalo pedagógico, porque **revela cuáles de tus jerarquías eran diseño y cuáles eran ceremonia**. Es el mismo experimento que hacen los dos manuales con los diagramas idénticos, pero aplicado a la herencia.

¿Cuándo la herencia sigue valiendo la pena en Python?

- ✓ Cuando comparte **implementación** - hay código real en el padre que las subclases reusan. Esta razón nunca fue sintáctica y no cambia.
- ✓ Cuando el **dominio afirma** el «es-un» - tu criterio del manual sigue intacto, palabra por palabra.
- ✓ Cuando querés que @abstractmethod te falle temprano - la ABC impide instanciar si falta una operación, y ese error al construir vale más que un AttributeError tres capas después.
- ✗ Cuando la única razón era «necesito un tipo común para el polimorfismo» → en Python eso es gratis, y la jerarquía es peso muerto que acopla sin dar nada.

El tercer punto de la lista -que la ABC falle temprano- se entiende mejor viendo los dos modos de fallo lado a lado. Con duck typing puro, el olvido de un método se descubre *al llamarlo*, en runtime, quizá en producción. Con la ABC, se descubre *al construir*, con un mensaje que nombra exactamente lo que falta:

PYTHON - DOS MODOS DE FALLO

```
\# Duck typing puro: el error espera hasta la llamada

class Pentagono: # olvidamos calcular_area()
    ...
\>>> f = Pentagono() # construye sin quejarse
\>>> f.calcular_area() # explota acá, quizá mucho después
AttributeError: 'Pentagono' object has no attribute 'calcular_area'
\# Con ABC: el error no espera

class Pentagono(Poligono): # Poligono es ABC con @abstractmethod
    ... # olvidamos lados_esperados()
\>>> f = Pentagono() # explota al construir
TypeError: Can't instantiate abstract class Pentagono without an
implementation for abstract method 'lados_esperados'
```

Los dos mensajes son reales, copiados del intérprete. La diferencia entre ellos es distancia: el `AttributeError` aparece donde se usa el objeto, que puede estar a tres módulos del error; el `TypeError` aparece donde se construye, que casi siempre está al lado. Ese acortamiento de distancia es lo que compras cuando elegís la ABC - y es una razón de diseño legítima, independiente del compilador.

Protocol: el contrato sin el acoplamiento

El duck typing puro tiene un costo evidente: no hay contrato. Nada documenta que el bucle espera un `calcular_area()`, y si le pasás un objeto que no lo tiene, te enterás en runtime.

Para eso está Protocol, que es tipado **estructural** verificable:

PYTHON

```
from typing import Protocol

class Dibujable(Protocol): # nadie hereda de esto

    def dibujar(self) -> None: ...

    def render(x: Dibujable) -> None: # mypy verifica que x tenga dibujar()
        x.dibujar()
```

La diferencia con una interfaz Java está en la dirección de la dependencia, y es más profunda de lo que parece. En Java, la clase **declara** que implementa la interfaz: `class Triangulo implements`

Dibujable. El que cumple el contrato tiene que conocer el contrato. En Python con Protocol, la clase **no se entera de que el protocolo existe**. Cumple con tener el método; el protocolo lo verifica desde afuera.

Esto tiene una consecuencia práctica enorme: podés tipar código de terceros. Si una librería te da una clase que tiene dibujar() pero no hereda de nada tuyo, en Java no podés hacer nada -no podés modificar su código para agregarle el implements- y terminás escribiendo un adaptador. En Python definís el Protocol y esa clase ajena ya lo cumple, retroactivamente, sin tocarla.

Protocol es, entonces, duck typing **verificable**: el contrato sin el acoplamiento del implements. Java 8 agregó default methods en interfaces, que se acercan a las ABCs con implementación - pero seguís necesitando declarar implements. Esa declaración es justamente lo que Python no pide.



Figura 9 · La flecha se invierte: en Java la clase declara la interfaz; con Protocol, el contrato verifica desde afuera y la clase no se entera.

Entonces, ¿ABC o Protocol?

La regla que funciona: **ABC cuando sos dueño de la jerarquía y hay implementación para compartir; Protocol cuando definís un contrato que otros cumplen sin coordinarse con vos**. El Poligono del Ejercicio 1 es una ABC: tiene código real que las subclases reusan y el dominio afirma la jerarquía. Un Dibujable que atraviesa módulos y que quizá cumplan clases de terceros es un Protocol. Y las dos cosas pueden convivir en el mismo sistema sin conflicto.

7 Regalos que Java no tiene

dataclass, herencia múltiple, operadores

Hasta acá el manual se ocupó de lo que se pierde o de lo que hay que repensar. Este capítulo va en la otra dirección: qué te da Python que en Java no tenés, o que en Java requiere trabajo. No es una lista de curiosidades - son herramientas que cambian cómo se escribe el modelo.

dataclass

Campos, `__init__`, `__eq__` y `repr` en tres líneas. Todo el boilerplate que en Java te genera el IDE y que después tenés que mantener a mano cada vez que agregás un campo:

PYTHON

```
from dataclasses import dataclass

@dataclass(frozen=True) # frozen = immutable de verdad
class Lado:
    longitud: float
    color: str = "negro"
    \# Lado(3.0) == Lado(3.0) → True
    \# Sin escribir equals(), hashCode(), toString().
```

La diferencia con el código generado por el IDE es que el generado es código: vive en tu archivo, ocupa cuarenta líneas, y el día que agregás un campo tenés que acordarte de regenerarlo o el `equals` queda mal en silencio. La `dataclass` no genera texto: deriva el comportamiento de la declaración de campos, así que **no se puede desincronizar**.

El `frozen=True` merece un párrafo aparte porque no tiene equivalente Java. Hace que la instancia sea immutable de verdad: asignar a un campo tira `FrozenInstanceError`. Java te da final campo por campo y aun así el objeto puede tener métodos que muten estructuras internas. El `frozen` es una propiedad de la clase entera, declarada en un lugar.

Herencia múltiple real

El Ejercicio 6 modela **overlapping**: un mismo empleado puede tener varios roles simultáneos. Java te obliga a resolverlo con interfaces o con composición, porque no permite heredar de dos clases. Python permite:

PYTHON

```
class Empleado(RolDocente, RolInvestigador, RolAdmin): ...
```

Funciona. El MRO -el algoritmo C3 que linealiza la jerarquía- resuelve el orden de resolución de métodos de forma determinista, y las tres clases aportan su implementación. Es la solución que un programador Java frustrado por años de no poder hacerlo va a querer usar el primer día.

CUIDADO - EL CRITERIO NO CAMBIÓ

Que Python *permita* herencia múltiple no la vuelve la mejor respuesta al overlapping. **La composición de roles opcionales que ya usás en el Ejercicio 6 sigue siendo el mejor modelo** - porque los roles se adquieren y se pierden en runtime, y la herencia es estática. Un empleado que deja de ser investigador tendría que cambiar de clase, lo cual es absurdo. Python te da la opción; el criterio de diseño no cambió.

Overlapping: dos formas de modelar los roles

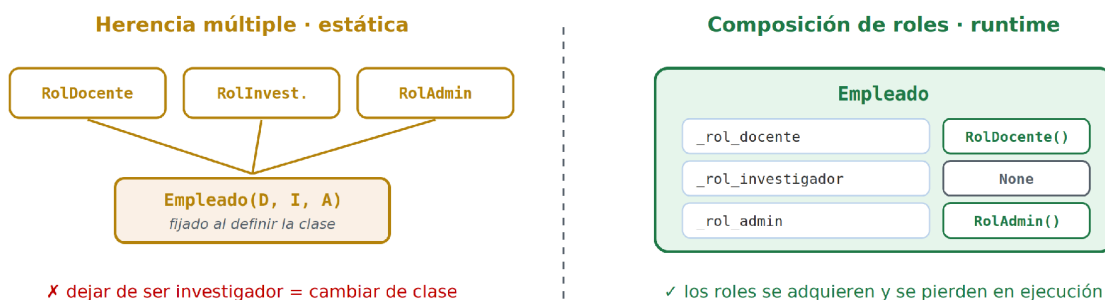


Figura 10 · El overlapping del Ejercicio 6: la herencia múltiple fija los roles al definir la clase; la composición los deja cambiar en ejecución.

Este caso es el mejor ejemplo del manual de que **la capacidad del lenguaje no es el criterio de diseño**. La pregunta nunca fue «¿puedo heredar de tres clases?». La pregunta era «¿los roles de un empleado son parte de su identidad permanente o son estados que cambian?». El enunciado responde que cambian. Con esa respuesta, la composición gana en Java y gana en Python, y que Python permita la alternativa es irrelevante.

Otros regalos

Recurso	Qué te da	Análogo Java
Sobrecarga de operadores	<code>__add__</code> , <code>__len__</code> , <code>__eq__</code> , <code>__lt__</code>	No existe
Context manager	<code>with</code> - <code>__enter__</code> / <code>__exit__</code>	<code>try-with-resources</code>
<code>@singledispatch</code>	Visitor sin boilerplate	Patrón Visitor completo
<code>@cached_property</code>	Derivado calculado una vez	Campo + lazy init a mano
Enum con comportamiento	Métodos y estado en el enum	Similar, más verboso
<code>__slots__</code>	Fija los atributos, ahorra memoria	Campos declarados (default)
Funciones de primera clase	Pasar métodos sin envolver	Lambdas / referencias a método

Dos de estos cambian cómo se ve el modelo. La **sobrecarga de operadores** permite que un Medida o un Dinero del dominio se sumen con `+`, que es como se escriben en el enunciado y en la cabeza del experto del dominio. Java te obliga a `a.sumar(b)`. Y `@cached_property` resuelve el atributo derivado costoso -el que en el UML lleva la barra pero que no querés recalcularlo cada vez- en una línea, sin el campo auxiliar y el `if (cache == null)` que Java necesita.

El operador merece verse una vez completo, porque combina tres cosas del capítulo en diez líneas - `dataclass`, inmutabilidad y `dunder`:

PYTHON - EL DOMINIO SE ESCRIBE COMO SE DICE

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Medida:
    valor: float
    unidad: str

    def __add__(self, otra: "Medida") -> "Medida":
        if self.unidad != otra.unidad:
            raise ValueError("unidades incompatibles")
        return Medida(self.valor + otra.valor, self.unidad)

>>> Medida(3, "m") + Medida(4, "m")
Medida(valor=7, unidad='m')
>>> Medida(3, "m") + Medida(4, "kg")
ValueError: unidades incompatibles
```


Notá que el `__add__` devuelve una Medida nueva en lugar de modificar la existente - obligado por el frozen, y correcto para un objeto-valor: dos medidas se suman como se suman dos números, produciendo un tercero. El *eq* y el `__repr__` de la salida vinieron gratis con la dataclass. En Java, este mismo objeto-valor son unas cuarenta líneas.

8 Checklist de desintoxicación

Los 7 java-ismos a buscar en tu código Python

Usalo como checklist de code review - tuya y de tus estudiantes. Si venís de Java, estos siete reflejos son los que producen «Java escrito en Python»: código que funciona, pasa los tests, y le grita al lector de dónde viene.

No están en orden de gravedad sino de frecuencia. El primero es, de lejos, el más común.

- | | | |
|---|---|--|
| 1 | ¿Escribí <code>get_x()</code> / <code>set_x()</code> ? | → Atributo público, o <code>@property</code> si hay lógica |
| 2 | ¿Puse <code>__doble_guion</code> creyendo que es <code>private</code> ? | → <code>_simple</code> es la convención |
| 3 | ¿Heredé de una clase base solo para tener un tipo común? | → Duck typing o Protocol |
| 4 | ¿Definí una interfaz vacía para hacer implements? | → Protocol |
| 5 | ¿Escribí <code>__init__</code> que solo asigna campos? | → <code>@dataclass</code> |
| 6 | ¿Traduje un stream con <code>for + append + acumulador</code> ? | → Comprehension |
| 7 | ¿Confío en que los type hints me protegen? | → Corré mypy, o no te protegen |

Cómo usar el checklist en clase

Los siete puntos tienen una estructura común que conviene explicitar cuando se enseña: cada uno es **un hábito que era correcto en Java**. Ninguno es ignorancia. El estudiante que escribe getters en Python no es que no sepa: es que sabe demasiado bien otra cosa. Esa es la diferencia entre corregir un error y desactivar un reflejo, y explica por qué el checklist no alcanza sin los capítulos que lo preceden. Marcar el punto 1 en un code review sin poder explicar el capítulo 2 produce obediencia, no comprensión - y el reflejo vuelve en el próximo archivo.

Cierre · Qué es UML y qué es sintaxis

El experimento controlado de los dos manuales

Los veintitrés diagramas idénticos entre el manual Java y el manual Python son el argumento entero de esta guía, y vale la pena decir por qué son un argumento y no una casualidad de producción.

Cuando dos manuales resuelven los mismos once enunciados en dos lenguajes distintos y llegan al mismo diagrama, tenés un **experimento controlado**. La variable independiente es el lenguaje; todo lo demás está fijo - el enunciado, el analista, el método de las tres lentes, los ocho patrones. Y el resultado es que el modelo no se movió un milímetro: clases, composición contra agregación, {XOR}, clase asociación, atributos derivados, generalización ortogonal, todo idéntico.

Lo que sí se movió fueron tres cosas, y ninguna es diseño:

- Los **modificadores**: private/final → convención, property y mypy. Cambió el mecanismo de cumplimiento, no la decisión de qué es interno.
- Las **ceremonias**: getters, equals, hashCode → property y dataclass. Desapareció el boilerplate, no el concepto de igualdad ni el de acceso controlado.
- La **obligatoriedad de la herencia**: requisito del compilador → decisión de dominio. La herencia no desapareció; dejó de ser gratuita y pasó a tener que justificarse.

El Ejercicio 7 es la prueba más limpia. La superclase deducida sobrevive al pasaje, y sobrevive *aunque Python no la necesite*. Si hubiera sido una concesión al compilador, en Python se habría evaporado. No se evaporó, porque nunca fue eso: era una afirmación sobre el dominio, y los dominios no cambian cuando cambiás de lenguaje. Lo que cae en Python es únicamente lo que en Java estaba ahí *para que compilara*.

Ahí está el valor pedagógico de tener las dos versiones. Un estudiante que solo vio el manual Java no tiene forma de distinguir qué parte de lo que aprendió era diseño y qué parte era Java: todo llega junto, en el mismo código, con la misma cara de necesario. Poner los dos manuales lado a lado, con el mismo diagrama arriba y dos códigos abajo, hace visible esa frontera. Lo que aparece en las dos versiones era modelo. Lo que aparece en una sola era lenguaje.

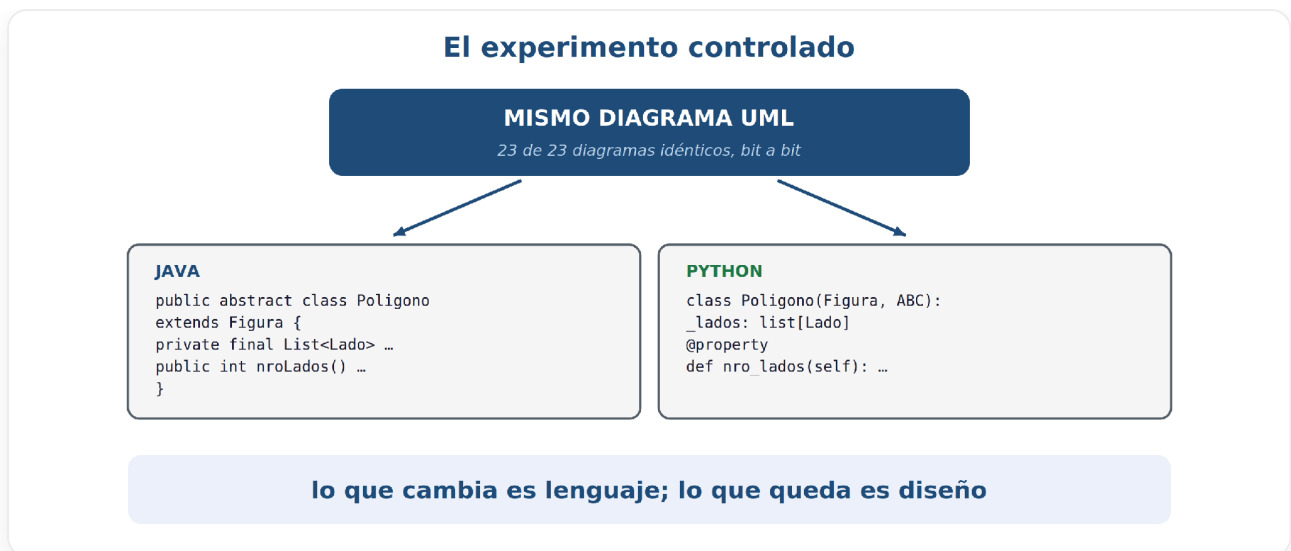


Figura 11 · El experimento: variable independiente el lenguaje, todo lo demás fijo. El diagrama no se mueve.

Poner los dos manuales lado a lado, con el mismo diagrama y dos códigos, es el experimento controlado:

lo que cambia es lenguaje; lo que queda es diseño.